Übungsblatt 5: Inter-Task-Kommunikation und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie wählen einen Kommunikationsvertrag, verfolgen Queue-Belegung, prüfen Pufferbesitz und rechnen eine Interruptübergabe nach. Grundlage sind Vorlesungen 1–5. FIFO bedeutet First In, First Out: Entnahme in Einfügereihenfolge.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	30 Orientierungspunkte
Transfer und Besprechung	20	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **85 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 75 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten. Planen Sie im Papierkern 20 Minuten für Aufgabe 1, 20 Minuten für Aufgaben 2–3 und 10 Minuten für Aufgabe 4.

Aufgabe 1: Ring und Wartezustände · 8 Punkte

Lehrkonvention: FIFO, $N = 3$, Head ist der nächste Entnahmeplatz, Tail der nächste Einfügeplatz, Count die Belegung. Alle drei Plätze sind nutzbar. Start: Head = Tail = Count = 0. Ein volles Try-Put verwirft den neuen Wert und ändert keine Queue-Daten. Ein leeres Try-Get liefert keine neue Nachricht. Try bedeutet Timeout null, ohne Warten.

1. Verfolgen Sie Put 10, Put 20, Get, Put 30 und Put 40. Geben Sie nach jedem Schritt logische Folge, Head, Tail und Count an. Geben Sie danach die physische Slotbelegung an. (3 P)
2. Setzen Sie fort: Put 50, Get, Put 50, Get, Get, Get, Get. Geben Sie je Operation Erfolg oder Fehler, einen gegebenenfalls gelieferten Wert und den neuen Zustand an. (3 P)
3. Welche zwei Belegungen können bei Head = Tail auftreten? Erklären Sie außerdem den Unterschied zwischen „Get wartet auf Daten“, „Put macht den Consumer Ready“ und „Consumer verarbeitet“. (2 P)

Musterlösung Aufgabe 1

Schritt	Ergebnis	Logische Folge	Head	Tail	Count
Start	–	leer	0	0	0
Put 10	Erfolg	10	0	1	1
Put 20	Erfolg	10, 20	0	2	2
Get	10	20	1	2	1
Put 30	Erfolg	20, 30	1	0	2
Put 40	Erfolg	20, 30, 40	1	1	3
Put 50	voll, Fehler	20, 30, 40	1	1	3
Get	20	30, 40	2	1	2
Put 50	Erfolg	30, 40, 50	2	2	3
Get	30	40, 50	0	2	2
Get	40	50	1	2	1
Get	50	leer	2	2	0
Get	leer, Fehler	leer	2	2	0

Nach dem ersten Put 40 liegen physisch `[40, 20, 30]` in den Slots. Die Folge beginnt bei Head, nicht zwingend bei Slot null. Gleiche Indizes können bei Count null leer und bei Count drei voll bedeuten. Alte Bytes in freigegebenen Slots sind keine gültigen Elemente.

Ein erlaubtes wartendes Get auf leerer Queue setzt den Consumer auf Blocked. Der erfolgreiche Put kann ihn Ready machen; erst die Auswahl durch den Scheduler lässt ihn Running werden und verarbeiten. Bei Timeout null blockiert der Get nicht. Ein abgelaufener oder fehlgeschlagener Get darf seinen Zielpuffer nicht als neue Nachricht verarbeiten.

Aufgabe 2: Überlauf und Kapazität · 8 Punkte

Lehrmodell, keine Messung: Queue anfangs leer, $N = 10$. Ein Put bei jeder ganzen Millisekunde von 1 bis 20 versucht den Wert seines Zeitpunkts einzureihen. Get bei 5, 10, 15 und 20 ms entnimmt je ein Element. Zeitgleich gilt **Put vor Get**. API-Zeit ist vernachlässigt. Volle Queue verwirft den neuen Datensatz.

1. Bestimmen Sie Belegung nach 5 und 10 ms und den ersten Fehl-Put. (2 P)

2. Bestimmen Sie verworfene Werte, entnommene Werte, Zahl erfolgreicher Puts und Restbelegung bei 20 ms. Prüfen Sie beide Nachrichtenbilanzen. (3 P)
3. Wie groß müsste die Queue sein, um nur dieses begrenzte Intervall verlustfrei zu speichern? Begründen Sie die höchste Zwischenbelegung. Kann eine endliche Vergrößerung bei dauerhaft denselben Raten jeden Verlust verhindern? (2 P)
4. Wählen Sie eine begründete Überlaufregel für einen Softwareauftrag, dessen Erzeugung warten darf, und für ein Display, das nur den neuesten Zustand braucht. (1 P)

Musterlösung Aufgabe 2

Nach 5 ms liegen vier, nach 10 ms acht Nachrichten vor. Bei 12 ms ist die Queue voll, der erste Fehl-Put liegt bei 13 ms. Verworfen werden 13, 14, 15, 17, 18, 19 und 20. Der Get bei 15 ms schafft erst nach dem dort abgelehnten Put einen Platz; deshalb gelingt Put 16. Die Gets liefern 1, 2, 3 und 4. Es gibt 13 erfolgreiche Puts, sieben Verluste und neun Restnachrichten.

Die Bilanzen lauten $20 = 13 + 7$ für Versuche, Erfolge und Verluste sowie $13 = 4 + 9$ für Erfolge, Entnahmen und Rest. Ein verlustfreier Referenzverlauf erreicht unmittelbar nach Put 20 und vor Get 20 die Belegung $20 - 3 = 17$. Daher sind 17 Plätze erforderlich, obwohl nach dem letzten Get nur 16 übrig wären.

Die angebotene Rate beträgt 1000 Nachrichten/s, die Entnahmerate 200 Nachrichten/s. Dauerhaft wächst der verlustfreie Rückstau um 800 Nachrichten/s. Ein endlicher Puffer verschiebt nur den Verlust. Der wartbare Softwareauftrag kann durch blockierendes Put Gegendruck erhalten, sofern kein benötigter Mutex gehalten wird. Das Display kann einen expliziten Latest-Value-Vertrag nutzen. Ein Try-Put auf einer vollen 1er-Queue lässt den alten Zustand stehen und erfüllt diesen Vertrag nicht.

Aufgabe 3: Kopie, Besitz und Pflichtempfänger · 6 Punkte

1. Eine Wertqueue kopiert einen Datensatz mit `sample = 10`. Der Sender ändert anschließend seine lokale Kopie auf 99. Was erhält der spätere Consumer? Warum gilt dieselbe Begründung nicht für den Inhalt hinter einem kopierten Pointer? (2 P)
2. Eine Queue bietet zehn reine Pointerplätze. Es existieren zwei Bildpuffer A und B mit je 75 KiB. A wird vom Consumer gelesen, B ist erfolgreich eingereiht. Wie viele neue unabhängige Bilder kann der Producer jetzt befüllen? Berechnen Sie Pointer-Slot-Nutzspeicher bei angenommenen 4-Byte-Pointern und den tatsächlich vorhandenen Bildspeicher. (2 P)
3. Ein Pointer-Put scheitert. Wer bleibt für den Puffer verantwortlich? Reicht ein `static`-Array als Schutz vor zu frühem Wiederbefüllen? Erhalten zwei Get-Consumer derselben Queue beide jede Nachricht? (2 P)

Musterlösung Aufgabe 3

Der spätere Consumer erhält zehn, sofern die gespeicherte Nachricht nicht anderweitig entnommen oder verworfen wurde. Die Queue hat die vereinbarten Bytes kopiert. Bei einem Pointer wird nur die Adresse kopiert; beide Teilnehmer beziehen sich auf dieselben externen Bytes. Deren Gültigkeit und unveränderte Verfügbarkeit brauchen einen eigenen Besitzvertrag.

Kein weiterer Bildpuffer ist frei. Zehn Pointerplätze ergeben $10 \cdot 4 = 40$ Byte Elementnutzspeicher. Zwei Bilder benötigen $2 \cdot 75 = 150$ KiB, also 153600 Byte. Kontrollblock, Alignment und Allokatorverwaltung kommen hinzu. Die reine Queue-Kapazität erzeugt keine zusätzlichen Bilder.

Nach fehlgeschlagenem Put bleibt der Producer verantwortlich. Er darf später erneut versuchen oder den Puffer genau einmal in den Free-Pool zurückgeben. `static` sichert Lebensdauer, aber

keine exklusive Nutzung. Zwei Get-Consumer verteilen Nachrichten: Ein Element wird nur einmal entnommen. Jeder Pflichtempfänger braucht einen eigenen Zustellvertrag, beispielsweise eine eigene Queue samt Fehlerbehandlung.

Aufgabe 4: Interruptübergabe und Zeitbudget · 8 Punkte

1. Lehrannahmen: vier implementierte NVIC-Prioritätsbits, passende Präemptionsgruppierung, logische Kernelaufrufgrenze fünf. Darf IRQ-Priorität drei `osMessageQueuePut(q, &m, 0, 0)` aufrufen? Wie steht es um Priorität sieben? Darf `osMutexAcquire(mutex, 0)` bei Priorität sieben aufgerufen werden? Berechnen Sie den geschobenen Registerwert der Grenze fünf und nennen Sie den Wert für den HAL-Prioritätssetter. (3 P)
2. Synthetische Spur in µs: Ereignis 0, ISR-Eintritt 3, Worker Ready 8, ISR-Ende 12, Workerstart 16, Abschluss 24. Bestimmen Sie Eintrittslatenz, ISR-Dauer, Ready-bis-Running, Startlatenz und Antwortzeit bis Abschluss. Prüfen Sie eine relative Deadline von 25 µs. Warum darf Ready-bis-Running nicht zusätzlich auf alle anderen Intervalle addiert werden? (3 P)
3. Ein anfangs leerer binärer Semaphor mit Maximum eins erhält drei Releases vor jedem Acquire. Wie viele Releases gelingen? Unterscheiden Sie diese Zusammenfassung von Tasterprellen und einem Queue-Drop. (2 P)

Musterlösung Aufgabe 4

Priorität drei ist dringlicher als die Grenze fünf und darf keine Kernel-API rufen. Priorität sieben liegt im zulässigen Bereich; Queue Put ist mit gültigem Handle und Timeout null aus einer erlaubten ISR vorgesehen. Mutex Acquire bleibt auch dort unzulässig. Die drei Bedingungen heißen erlaubte Funktion, nicht wartender Parameter und passende IRQ-Priorität.

Der Registerwert ist $5 \cdot 2^{8-4} = 80 = 0\text{x}50$. Der HAL-Setter erhält die logische fünf, nicht 0x50. Diese Zahlen gelten nur unter den gegebenen Annahmen.

Eintrittslatenz: $3 - 0 = 3$ µs. ISR-Dauer: $12 - 3 = 9$ µs. Ready-bis-Running: $16 - 8 = 8$ µs. Startlatenz: $16 - 0 = 16$ µs. Antwortzeit bis Abschluss: $24 - 0 = 24$ µs. Die Frist 25 µs wird mit 1 µs Restbudget erfüllt. Ready-bis-Running überlappt einen Teil der ISR und den Übergang. Alternativ zerlegt man ohne Überlappung $3 + 9 + 4 + 8 = 24$ µs. Der letzte Summand ist Workerarbeit, nicht zusätzliches Ready-Warten.

Ein Release gelingt, zwei melden fehlende Kapazität. Das ist Zusammenfassung mehrerer Arbeitshinweise. Prellen erzeugt mehrere elektrische Flanken; ein Queue-Drop lehnt eine bereits erfasste Nachricht wegen Vollbelegung ab. Hardware-Pendingbits können schon vor dem Callback zusammenfassen. Keine dieser Zählungen ist automatisch die Zahl entprellter Betätigungen.

Transfer: Zwei Nachrichten, ein Speicher · 20 Minuten

Bearbeiten Sie den Fall zehn Minuten und besprechen Sie die Besitzregeln zehn Minuten. Keine zusätzlichen Punkte. Das folgende **abstrakte Ablaufmodell** hat keine gleichzeitigen Zugriffe: Der Consumer beginnt erst nach allen Producer-Schritten. Die Queue besitzt zwei Plätze, alle Puts gelingen.

Schritt	Producer / Queue
1	statisches Array `buf` enthält `[10, 20]`
2	Pointer auf `buf` wird eingereiht
3	Producer ändert dasselbe Array auf `[30, 40]`

Schritt	Producer / Queue
4	derselbe Pointer wird nochmals eingereiht
5	Consumer entnimmt beide Pointer und liest die jeweiligen Arrays

1. Welche beiden Inhalte liest der Consumer? Warum sind zwei Queueplätze keine zwei unabhängigen Bildpuffer?
2. Vergleichen Sie mit einer Queue, die bei jedem Put die beiden Zahlen **als Werte** kopiert. Welche Inhalte werden dann entnommen?
3. Entwerfen Sie für die Pointervariante einen Besitzablauf mit zwei verschiedenen Puffern und Free-Pool. Was geschieht nach fehlgeschlagenem Put? Reicht `static` zur exklusiven Nutzung?

Musterlösung Transfer 5

Beide Pointer bezeichnen dasselbe Array; zweimal wird `[30, 40]` gelesen. Der erste Bildstand ist überschrieben. Es gibt nur einen Datenpuffer, obwohl zwei Adressen gespeichert sind. Im gegebenen Ablauf liegt kein gleichzeitiger C-Zugriff vor; der Fehler ist bereits durch falsche Wiederverwendung erklärbar.

Die Wertqueue liefert `[10, 20]` und `[30, 40]`, weil sie jeweils die Datensatzbytes kopiert. Der Pointervertrag braucht dagegen zwei freie eigenständige Puffer: Producer nimmt einen, füllt ihn und übergibt ihn bei erfolgreichem Put. Consumer entnimmt, verarbeitet und gibt ihn genau einmal frei. Vor der Rückgabe darf Producer ihn nicht überschreiben. Bei fehlgeschlagenem Put bleibt der Producer zuständig und kann erneut versuchen oder zurückgeben. `static` sichert Lebensdauer, keinen Besitz.

Praxisfall · 15 Minuten

Praxisfall 5: Einen abgeschlossenen Queue-Lauf prüfen

Papieralternative, synthetischer Diagnosefall: Eine FIFO-Wertqueue hat einen Producer, einen Consumer und keine anderen Entnahmen. Angeboten werden Sequenzen 1–20 mit Nutzwert $10 \cdot \text{seq}$. Angenommen wurden 1, 2, 3, 4, 5, 9, 13 und 17; alle anderen Puts wurden ausschließlich wegen Vollbelegung verworfen. Eine separate STOP-Nachricht folgt hinter den angenommenen Daten. Beide Done-Kennzeichen sind gesetzt, alle Daten verarbeitet und die Queue leer. Das Consumerlog enthält dieselbe Folge mit den passenden Nutzwerten.

1. Bestimmen Sie `accepted`, `dropped` und `received` und prüfen Sie beide Bilanzen. Wird STOP als Datenversuch mitgezählt?
2. Welche Endsequenzen fehlen? Kann das Consumerlog allein diese Verluste erkennen, wenn es die Gesamtzahl der Angebote nicht kennt?
3. Ein anderer angeblich gültiger Endlog nennt 1, 2, 4, 3, 5, 9, 13, 17. Ist das mit diesem Kanalvertrag vereinbar? Begründen Sie, bevor Sie die Ursache einem Scheduler zuschreiben.

Bei vorbereitetem Board: Die Lehrperson zeigt einen bereits abgeschlossenen Wertqueue-Lauf und dessen Bilanzen. Ein eigener Button-/EXTI-Versuch ist Vertiefung; er wird für die Erklärung des Queue-Vertrags nicht benötigt.

Musterlösung Praxisfall 5

Acht Annahmen, zwölf Drops, acht empfangene Daten. $20 = 8 + 12$ und nach Abschluss $8 = 8 + 0$. STOP wird separat behandelt und ist kein Datenversuch. Unter den gegebenen Bedingungen

fehlt nach 17 auch 18, 19 und 20. Ohne Wissen über alle Angebote ist dieser hintere Verlust aus dem Consumerlog allein nicht erkennbar.

Die vertauschten Sequenzen 4 und 3 widersprechen der Einfügereihenfolge bei einem Producer, einem Consumer und einer FIFO-Wertqueue. Scheduling kann die Ausführungszeit beeinflussen, ändert aber diesen Vertrag nicht. Der Log- oder Kanalvertrag muss überprüft werden. Dies sind vorgegebene Papierdaten, keine vorhergesagten realen Dropzahlen.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (30 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 5; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*; die in der Vorlesung und der freiwilligen Integrationsanleitung genannten Herstellerdokumente.